

ECOM EXPERTS

Ecom combifilters

Manual de usuario, técnico y de desarrollo

ecom_combifilters · 08/09/2026

Contenido

¿Qué hace este módulo?

Instalación

Configuración

Cómo lo ve tu cliente

Requisitos previos del producto

Preguntas frecuentes

Soporte

1. Propósito

2. Problema que resuelve

3. Arquitectura

4. Hooks

5. Flujo de datos

6. Estrategia SQL (clave del rendimiento)

7. Seguridad

8. Configuración (Configuration)

9. Registro Gmartos

10. Depuración

1. Convenciones del proyecto

2. Puntos de extensión

2.1 Añadir un nuevo action al AJAX

2.2 Nuevas consultas

2.3 Cambiar dónde se pinta el wizard

3. Contrato del endpoint AJAX

4. Estado en el front (combifilters.js)

5. Añadir al carrito

6. Rendimiento y escalado

7. Pruebas recomendadas

8. Mantenimiento

Versión: 1.0.0

¿Qué hace este módulo?

Permite vender productos con **muchísimas combinaciones** (cientos o miles: pinturas con 4000 colores, formatos, acabados...) mostrando un **asistente (wizard)** en la ficha de producto. El cliente busca, filtra y elige su combinación cómodamente, y la añade al carrito. Todo sin que la página se cargue lenta.

Instalación

1. Copia la carpeta `ecom_combifilters` en `/modules` de tu tienda.
2. Ve a **Módulos** → **Gestor de módulos**, busca "Ecom CombiFilters" e instala.
3. Pulsa **Configurar**.

Configuración

Opción	Para qué sirve
Activar wizard	Enciende o apaga el módulo.
Umbral de combinaciones	El wizard solo aparece si el producto tiene más combinaciones que este número. Por defecto 50. Pon 0 para que aparezca siempre.
Modo de interfaz	<i>Wizard por pasos</i> (un atributo cada vez) o <i>Panel de filtros</i> (todos a la vez).
Posición del wizard	Dónde aparece en la ficha: <i>En actions</i> (junto al botón comprar), <i>Debajo de añadir al carrito</i> , <i>Bajo buttons</i> , o <i>Central</i> (pestaña de contenido extra).
Resultados por página	Cuántos valores/combinaciones se cargan de golpe.
Ocultar selector nativo	Esconde el selector original del tema para no duplicar.
Habilitar búsqueda	Muestra una caja de búsqueda en cada atributo (ideal para miles de colores).
Mostrar imágenes	Muestra la foto de cada combinación.
Mostrar stock	Indica si hay existencias y bloquea el botón si no hay.
Activar logs	Guarda registros técnicos para soporte (solo si te lo piden).

Cómo lo ve tu cliente

1. Entra en un producto con muchas combinaciones.

2. Ve el bloque "**Configura tu producto**" con cada atributo (Color, Formato, Acabado...).
3. En atributos con muchos valores (ej. 4000 colores) escribe en la **búsqueda** para encontrar el suyo.
4. Al ir eligiendo, el wizard **descarta automáticamente** las opciones que no existen juntas.
5. Cuando la elección es única, aparece el **precio, imagen y stock**.
6. Indica la **cantidad** y pulsa **Añadir al carrito**.

Requisitos previos del producto

- El producto debe tener sus **combinaciones creadas** en PrestaShop (Catálogo → Producto → Combinaciones).
- Cada combinación debe tener su precio/stock como siempre. El módulo no crea combinaciones, solo las presenta mejor.

Preguntas frecuentes

No aparece el wizard. Comprueba que el producto supera el *Umbral de combinaciones* y que el módulo está activo. Pon el umbral a 0 para probar.

Salen dos selectores. Activa *Ocultar selector nativo*.

Va lento con muchos colores. Baja *Resultados por página* (ej. 20) y asegúrate de tener la búsqueda activada.

¿Funciona con cualquier nº de atributos? Sí. 3 atributos × 3 valores, 10 × 30, 30 × 40... el wizard se adapta solo.

Soporte

Email: ecomseo@gmail.com — Indica si te pedimos activar los *logs* y envía los `.log` de la carpeta `/logs` del módulo.

Ecom CombiFilters — Documentación Técnica

Versión: 1.0.0 · **Compatibilidad:** PrestaShop 1.7.6 – 9.x · **PHP:** 7.4 – 8.x · **Licencia:** AFL-3.0

1. Propósito

Reemplaza el selector de combinaciones nativo en productos con **miles de combinaciones** (probado para 8000+) por un wizard que filtra, busca y añade al carrito sin cargar todas las combinaciones en el HTML.

2. Problema que resuelve

El tema clásico de PrestaShop serializa **todas** las combinaciones en un objeto JS dentro de la página de producto. Con 8000 combinaciones eso son varios MB de JSON, parseo lento y navegador bloqueado. CombiFilters no precarga nada: el front pide datos por AJAX, paginados y filtrados en BD.

3. Arquitectura

```
ecom_combifilters/  
├─ ecom_combifilters.php           Clase principal (hooks, config, Gmartos)  
├─ classes/  
│   ├─ CombiFiltersLogger.php     Logger condicional -> /logs  
│   └─ CombiFiltersQuery.php      Núcleo SQL (conteo, grupos, valores, resolución)  
├─ controllers/front/ajax.php      Endpoint AJAX (values | resolve | detail)  
├─ views/  
│   ├─ templates/hook/wizard.tpl   HTML del wizard  
│   └─ templates/admin/gmartos_update.tpl  
│   ├─ js/combifilters.js         Lógica de UI (vanilla JS, sin jQuery)  
│   └─ css/combifilters.css  
├─ docs/                          Esta documentación  
└─ logs/                          Logs si ECOM_CF_DEBUG = 1
```

4. Hooks

Hook	Uso
<code>actionFrontControllerSetMedia</code>	Registra CSS/JS y <code>Media::addJsDef</code> solo en producto y si supera el umbral.
<code>displayProductActions</code>	Posición <code>actions</code> (junto al botón comprar).
<code>displayProductAdditionalInfo</code>	Posición <code>below_cart</code> (debajo de añadir al carrito).
<code>displayProductButtons</code>	Posición <code>buttons</code> (bajo buttons).
<code>displayProductExtraContent</code>	Posición <code>central</code> (pestaña de contenido extra; devuelve <code>ProductExtraContent</code>).

La posición se elige en `ECOM_CF_POSITION` . Se registran los 4 hooks pero cada uno solo pinta si coincide con la posición configurada (`renderWizardAt`).

5. Flujo de datos

1. **Render (PHP):** el hook solo imprime los *grupos* de atributos (nombre + nº de valores). No imprime valores ni combinaciones.
2. **Init (JS):** por cada grupo se llama `action=values` → valores compatibles paginados.
3. **Selección:** al elegir un valor se guarda en `selected{ id_group: id_attribute }` , se recargan los demás grupos (recalcula compatibilidad) y se llama `action=resolve` .
4. **Resolución:** `resolve` devuelve el nº total de combinaciones coincidentes + una página de detalles. Si total = 1 → selección automática para carrito.
5. **Carrito:** POST al controlador `cart` nativo (`add=1` , `id_product_attribute` , `qty` , `token`) y `prestashop.emit('updateCart')` .

6. Estrategia SQL (clave del rendimiento)

Subconsulta central — combinaciones que contienen TODOS los `id_attribute` seleccionados:

```
SELECT pac.id_product_attribute
FROM ps_product_attribute pa
INNER JOIN ps_product_attribute_combination pac
    ON pac.id_product_attribute = pa.id_product_attribute
WHERE pa.id_product = :idp
    AND pac.id_attribute IN (:seleccionados)
GROUP BY pac.id_product_attribute
HAVING COUNT(DISTINCT pac.id_attribute) = :num_seleccionados
```

- **Valores compatibles de un grupo** (`getValues`): aplica la subconsulta anterior con la selección de *otros* grupos y agrupa por `id_attribute` , con `LIKE` para búsqueda y `LIMIT/OFFSET` para paginación (soporta

4000+ colores).

- **Resolución** (`resolve`): cuenta total con `COUNT(*)` sobre la subconsulta y trae solo la página visible.
- Todas las consultas se apoyan en índices nativos de `product_attribute_combination` (`id_product_attribute` , `id_attribute`).
- Precio vía `Product::getPriceStatic()` (respeta impuestos, specific prices, reglas). Stock vía `StockAvailable` .

7. Seguridad

- Token de cabecera (`Tools::getToken(false)`) validado en cada llamada AJAX. Sin session locking.
- `pSQL()` en toda entrada de texto (búsqueda). IDs casteados a `(int)` .
- `if (!defined('_PS_VERSION_')) exit;` en todos los PHP. `index.php` en cada directorio.
- Sin `LIMIT 1` manual en `getValue` / `getRow` . Salida JSON escapada en JS con `escapeHtml` .

8. Configuración (Configuration)

Clave	Default	Descripción
<code>ECOM_CF_ENABLED</code>	1	Activa el wizard.
<code>ECOM_CF_THRESHOLD</code>	50	Mínimo de combinaciones para activarlo (0 = siempre).
<code>ECOM_CF_MODE</code>	stepper	<code>stepper</code> o <code>filters</code> .
<code>ECOM_CF_POSITION</code>	actions	Dónde se pinta: <code>actions</code> , <code>below_cart</code> , <code>buttons</code> , <code>central</code> .
<code>ECOM_CF_PER_PAGE</code>	30	Resultados/valores por página.
<code>ECOM_CF_HIDE_NATIVE</code>	1	Oculto <code>.product-variants</code> del tema.
<code>ECOM_CF_SEARCH</code>	1	Caja de búsqueda por grupo.
<code>ECOM_CF_SHOW_IMAGES</code>	1	Imagen por combinación.
<code>ECOM_CF_SHOW_STOCK</code>	1	Stock por combinación.
<code>ECOM_CF_DEBUG</code>	0	Logs en <code>/logs</code> .

9. Registro Gmartos

`checkGmartosUpdate()` registra el dominio/versión en `modules.gmartos.es` (ping semanal, timeout 3s, fallo silencioso). **Implementado con cURL nativo** (no Guzzle) para cumplir la regla CERO-Composer.

10. Depuración

Activa `ECOM_CF_DEBUG` . Se generan `logs/combifilters-YYYY-MM-DD.log` con cada render y llamada AJAX (acción, selección, grupo, búsqueda, página).

Ecom CombiFilters — Guía de Desarrollo

Versión: 1.0.0 · Para desarrolladores que extienden o mantienen el módulo.

1. Convenciones del proyecto

- Clase principal `Ecom_Combifilters` (coincide con carpeta `ecom_combifilters`), **sin namespace** (global namespace).
- Carga manual con `require_once`. **Cero Composer** (sin Guzzle/Symfony).
- Cabecera AFL-3.0 en todos los ficheros, `if (!defined('_PS_VERSION_')) exit;` tras la cabecera.
- `is_object($controller) ? get_class($controller) : ''` en vez de `::class` para máxima compatibilidad.
- `\Context::getContext()` con backslash.

2. Puntos de extensión

2.1 Añadir un nuevo `action` al AJAX

En `controllers/front/ajax.php`, dentro de `postProcess()` → `switch ($action)`, añade un `case` y un método `actionX()`. Recuerda validar token e `id_product` (ya se hace antes del switch).

2.2 Nuevas consultas

Toda la lógica SQL vive en `classes/CombiFiltersQuery.php`. Reutiliza `buildMatchingSubquery($id_product, $selectedAttributes)` para filtrar por combinaciones que contienen un conjunto de valores. Devuelve `null` si no hay selección (no restringir).

2.3 Cambiar dónde se pinta el wizard

Cambia el hook `displayProductActions` por otro (`displayProductAdditionalInfo`, `displayAfterProductThumbs`, etc.) en: - `install()` array de hooks, - `addnewfeatures()` array de hooks, - renombra `hookDisplayProductActions` acorde.

3. Contrato del endpoint AJAX

GET/POST {getModuleLink ajax} CON `token`, `id_product`, `ajax=1`, `action`.

`action=values` → { `id_group`, `selected` (JSON), `search`, `page`, `per_page` }

```
{ "success": true, "total": 4000, "page": 1, "per_page": 30,
  "values": [ { "id_attribute": 12, "name": "Rojo", "color": "#ff0000", "combo_count": 120 } ] }
```

`action=resolve` → { `selected` (JSON), `page`, `per_page` }

```
{ "success": true, "total": 1, "page": 1, "per_page": 30,
  "combinations": [ { "id_product_attribute": 9931, "reference": "RAL-3020-1L",
    "label": "Color: Rojo, Formato: 1L", "price": 12.5, "price_formatted": "12,50 €",
    "quantity": 30, "available": true, "image": "https://.../9931.jpg" } ] }
```

`action=detail` → `{ id_product_attribute }` → `{ success, combination }`.

4. Estado en el front (combifilters.js)

- `selected = { id_group: id_attribute }` — verdad única de la selección.
- `selectedJson(excludeGroup)` — serializa excluyendo un grupo (para recalcular ese grupo).
- Al elegir un valor: marca → recarga *otros* grupos → `resolveCombinations(true)`.
- `setCart(combination)` habilita el botón de carrito; `null` lo oculta.

5. Añadir al carrito

Se usa el controlador `cart` nativo (no se reinventa). Parámetros: `controller=cart`, `add=1`, `action=update`, `ajax=1`, `id_product`, `id_product_attribute`, `qty`, `token`. El token es `prestashop.static_token` (= `Tools::getToken(false)`). Tras éxito: `prestashop.emit('updateCart', {...})` para refrescar el bloque de carrito del tema.

6. Rendimiento y escalado

- No cargar nunca todas las combinaciones en PHP ni en JS. Siempre paginar/filtrar en BD.
- Si un producto tiene millones de combinaciones, sube índices personalizados sobre `product_attribute_combination(id_attribute, id_product_attribute)` (PrestaShop ya los trae por defecto).
- `ECOM_CF_PER_PAGE` controla el tamaño de página; bájalo si la latencia sube.

7. Pruebas recomendadas

1. Producto con 3 atributos × 3 valores (9 combos) — el wizard debe resolver a 1 combo al elegir los 3.
2. Producto con 1 atributo de 4000 valores — la búsqueda y paginación deben responder < 300 ms.
3. Combinación sin stock — botón de carrito deshabilitado si `ECOM_CF_SHOW_STOCK`.
4. Selección parcial — debe listar combinaciones coincidentes y permitir elegir una.
5. Activar `ECOM_CF_DEBUG` y revisar `/logs`.

8. Mantenimiento

- Sin ficheros `upgrade`: `addnewfeatures()` (en `getContent`) re-registra hooks faltantes con `try{}`.

- Al añadir hooks/clases, regenerar el JSON MCP del módulo.